

Proyecto Final de Programación

Luis D. Rodríguez López

Programación Orientado a Objetos, Tecnológico de Costa Rica

Mauricio Aviles Cisneros

25 de junio de 2026

a. Introducción

El presente proyecto consiste en el desarrollo de una aplicación de escritorio para el control de préstamos de ítems. La aplicación fue realizada en Java, aplicando conceptos de programación orientada a objetos, separación de responsabilidades, uso de clases, relaciones entre objetos, colecciones dinámicas, interfaz gráfica y persistencia de datos.

El objetivo principal del sistema es permitir que una persona o institución pueda administrar objetos que pueden ser prestados, registrar las personas que reciben préstamos, controlar cuáles ítems se encuentran prestados, retornar ítems, finalizar préstamos y generar reportes según diferentes criterios.

El proyecto incluye una capa lógica, una capa de control y una interfaz gráfica desarrollada con Java Swing. Además, se incorporó un mecanismo de guardado y carga de datos mediante serialización, lo que permite conservar la información registrada aunque el programa se cierre.

b. Descripción del problema

i. Qué es lo que hay que resolver

El problema que se busca resolver es la falta de control sobre ítems que son prestados a diferentes personas. En muchos casos, si no existe un sistema organizado, puede ser difícil saber qué objetos están disponibles, cuáles están prestados, quién los tiene, a qué categoría pertenecen o qué tipo de ítem son.

Por esta razón, se necesitaba crear una aplicación que permitiera registrar la información principal del sistema y mantener un control claro de los préstamos activos. Además, el sistema debía permitir generar reportes para consultar la información de manera más ordenada.

ii. Lista de subproblemas y explicación de cada uno

- El primer subproblema fue la administración de tipos. Cada ítem debe tener un tipo asociado, por ejemplo libro, herramienta, equipo u otro. Por eso, el sistema debía permitir crear, modificar, borrar y consultar tipos.

- El segundo subproblema fue la administración de categorías. Las categorías permiten agrupar ítems según una clasificación más específica. Un ítem puede pertenecer a cero o más categorías, por lo que el sistema debía permitir registrar categorías y asociarlas a los ítems.
- El tercer subproblema fue la administración de personas. Para poder realizar préstamos, era necesario registrar a las personas que pueden recibir ítems. Cada persona cuenta con datos como nombre, teléfono y correo electrónico.
- El cuarto subproblema fue la administración de ítems. El sistema debía permitir registrar objetos prestables con código, nombre, descripción, tipo y categorías asociadas. También debía validar que no se registraran ítems duplicados y que el tipo o las categorías indicadas existieran previamente.
- El quinto subproblema fue la creación de préstamos. El sistema debía permitir prestar uno o varios ítems a una persona, siempre que esos ítems existieran y no estuvieran prestados en otro préstamo activo.
- El sexto subproblema fue el control de disponibilidad de los ítems. El sistema debía evitar que un mismo ítem fuera prestado dos veces al mismo tiempo.
- El séptimo subproblema fue el retorno de ítems. El sistema debía permitir retornar un ítem específico dentro de un préstamo sin necesidad de finalizar todo el préstamo.
- El octavo subproblema fue la finalización de préstamos. Cuando todos los ítems de un préstamo se devuelven o cuando se desea cerrar el préstamo completo, el sistema debía permitir finalizarlo.
- El noveno subproblema fue la administración de alertas. Cada préstamo podía tener una alerta asociada con un mensaje, un tiempo de activación, una fecha de inicio y la indicación de si era recurrente o no.

- El décimo subproblema fue la generación de reportes. El sistema debía permitir consultar reportes por usuario, por ítem, por categoría y por tipo, para mostrar la información de manera más organizada.
- El último subproblema fue la persistencia de datos. La información no debía perderse al cerrar el programa, por lo que fue necesario implementar un mecanismo para guardar y cargar los datos automáticamente.

c. Metodología

i. Explicación técnica de cómo se resolvió cada subproblema

Para resolver el proyecto, se dividió el sistema en tres partes principales: lógica, control e interfaz gráfica.

En la parte lógica se crearon las clases que representan los elementos principales del sistema. Entre ellas se encuentran Tipo, Categoría, Persona, Ítem, Préstamo y Alerta. Cada clase contiene sus atributos privados, constructores, métodos de acceso, métodos modificadores y métodos relacionados con su comportamiento.

La administración de tipos se resolvió mediante la clase Tipo y una lista de tipos dentro del controlador. El sistema permite crear un tipo, modificarlo, consultarlo, borrarlo y listar todos los tipos existentes.

La administración de categorías se resolvió de forma similar, utilizando la clase Categoría. Cada categoría puede tener ítems asociados. Además, cuando un ítem se asocia a una categoría, también se actualiza la relación desde la categoría hacia el ítem.

La administración de personas se resolvió con la clase Persona. Cada persona tiene nombre, teléfono, correo electrónico y una lista de préstamos asociados. El correo electrónico se utiliza como identificador para buscar, modificar o eliminar personas.

La administración de ítems se resolvió con la clase Item. Cada ítem tiene un código, nombre, descripción, tipo, lista de categorías y lista de préstamos asociados. El código funciona como identificador único. Antes de crear un ítem, el controlador revisa que el código no exista, que el tipo indicado exista y que las categorías indicadas también existan.

La creación de préstamos se resolvió con la clase Prestamo. Un préstamo contiene una persona prestataria, una lista de ítems prestados, una alerta asociada y una fecha de préstamo. Antes de crear un préstamo, el controlador valida que la persona exista, que los ítems existan y que no estén prestados actualmente.

El control de disponibilidad de los ítems se resolvió revisando la lista de préstamos activos. Si un ítem ya se encuentra dentro de un préstamo activo, el sistema no permite prestarlo nuevamente.

El retorno de ítems se resolvió eliminando un ítem específico de la lista de ítems prestados dentro de un préstamo. Al eliminarlo, también se actualiza la relación del ítem con el préstamo.

La finalización de préstamos se resolvió eliminando todos los ítems del préstamo, quitando la relación del préstamo con la persona y removiendo el préstamo de la lista de préstamos activos.

Las alertas se resolvieron mediante la clase Alerta, la cual contiene el mensaje, si es recurrente, el tiempo de activación y la fecha de inicio. Esta alerta se asocia a cada préstamo al momento de crearlo.

Los reportes se resolvieron en el controlador mediante métodos que recorren las listas del sistema y construyen cadenas de texto con la información correspondiente. Existen reportes por usuario, por ítem, por categoría y por tipo.

La persistencia de datos se resolvió utilizando serialización. Para esto, las clases principales implementan Serializable. El controlador se guarda en un archivo llamado datos_prestamos.dat, y al abrir el programa se intenta cargar la información desde ese archivo. Si el archivo no existe, el sistema inicia con datos vacíos.

La interfaz gráfica se desarrolló con Java Swing. Se creó una ventana principal con pestañas, donde cada pestaña permite administrar una parte del sistema: tipos, categorías, personas, ítems, préstamos y reportes.

ii. Explicación del diagrama de clases detallado

El diagrama de clases representa las clases principales del sistema y sus relaciones.

La clase Tipo representa la clasificación principal de un ítem. Un tipo puede estar asociado a varios ítems, pero cada ítem tiene un solo tipo.

La clase Categoria representa una agrupación adicional de los ítems. Un ítem puede tener cero o más categorías, y una categoría puede contener varios ítems.

La clase Item representa los objetos que pueden ser prestados. Cada ítem posee un código único, nombre, descripción, tipo, categorías asociadas y una relación con los préstamos en los que ha participado.

La clase Persona representa a los usuarios que pueden recibir préstamos. Cada persona tiene nombre, teléfono, correo electrónico y una lista de préstamos asociados.

La clase Prestamo representa el préstamo de uno o varios ítems a una persona. Contiene la persona prestataria, los ítems prestados, una alerta y la fecha del préstamo.

La clase Alerta representa un aviso asociado a un préstamo. Contiene el mensaje de la alerta, si es recurrente, el tiempo de activación y la fecha de inicio.

La clase PrestamoControlador funciona como intermediaria entre la interfaz gráfica y la lógica del sistema. Esta clase contiene las listas principales de tipos, categorías, personas, ítems y préstamos. Además, contiene los métodos para crear, modificar, borrar, consultar y generar reportes.

La relación entre las clases permite que el sistema mantenga organizada la información. Por ejemplo, cuando se crea un préstamo, este queda asociado a una persona y a varios ítems. De igual forma, cuando se retorna un ítem o se finaliza un préstamo, se actualizan las relaciones correspondientes.

iii. Colecciones, bibliotecas, patrones y algoritmos utilizados

Para almacenar la información del sistema se utilizaron colecciones de tipo ArrayList. Estas colecciones permiten manejar listas dinámicas de tipos, categorías, personas, ítems y préstamos. Entre las bibliotecas utilizadas se encuentra java.util.ArrayList para las colecciones, javax.swing para la interfaz gráfica, java.awt para algunos elementos

visuales y java.io para la serialización y manejo de archivos. El proyecto utiliza una separación similar al patrón Modelo-Vista-Controlador. La capa lógica representa el modelo, la interfaz gráfica representa la vista y la clase PrestamoControlador funciona como controlador. Aunque no se implementa un MVC completo de forma estricta, sí se aplicó la idea de separar responsabilidades para que la interfaz no maneje directamente toda la lógica interna. También se aplicó encapsulamiento, ya que los atributos de las clases se mantienen privados y se acceden mediante métodos públicos. Esto permite proteger los datos y controlar la forma en que se modifican. Los algoritmos utilizados son principalmente búsquedas secuenciales sobre listas. Por ejemplo, para consultar un ítem se recorre la lista de ítems hasta encontrar el código solicitado. Lo mismo ocurre al buscar personas por correo, tipos por nombre o categorías por nombre. También se utilizan validaciones antes de realizar operaciones importantes. Por ejemplo, antes de crear un préstamo se verifica que la persona exista, que los ítems existan y que no estén prestados. Antes de crear un ítem se verifica que el código no esté repetido y que el tipo y las categorías sean válidos.

d. Análisis crítico de la implementación

La implementación logró cumplir con las funcionalidades principales solicitadas. Se desarrolló una aplicación con interfaz gráfica que permite administrar tipos, categorías, personas, ítems, préstamos, alertas y reportes. Además, el sistema guarda y carga los datos para no perder la información al cerrar el programa

Uno de los aspectos positivos de la implementación es la separación entre la lógica del sistema, el controlador y la interfaz gráfica. Esto facilita entender el código y evita que toda la lógica quede dentro de las ventanas.

Otro aspecto positivo es el uso de relaciones entre objetos. Por ejemplo, una persona conoce sus préstamos, un préstamo conoce sus ítems y un ítem conoce sus categorías y préstamos relacionados. Esto permite generar reportes y mantener la información conectada.

También se incorporaron validaciones importantes, como evitar códigos repetidos, evitar préstamos de ítems ya prestados y evitar crear ítems con tipos o categorías inexistentes.

Sin embargo, hay aspectos que podrían mejorarse. La interfaz gráfica funciona, pero podría ser más amigable si se usaran tablas, listas desplegables o componentes más avanzados. Por ejemplo, en lugar de escribir manualmente el nombre de un tipo o categoría, se podría seleccionar desde un JComboBox.

Otro aspecto mejorable es el manejo de fechas. Actualmente las fechas se manejan como texto, pero sería mejor utilizar clases como `LocalDate` para validar fechas y trabajar con cálculos de tiempo.

También se podría mejorar la identificación de préstamos. En la implementación actual se utiliza la posición del préstamo en la lista para algunas operaciones, pero sería más adecuado que cada préstamo tuviera un identificador único.

La persistencia mediante serialización funciona para un proyecto académico, pero en un sistema más grande sería mejor utilizar una base de datos. Esto permitiría consultar, modificar y respaldar información de forma más robusta.

e. Demostración del sistema implementado

Para demostrar el sistema, primero se ejecuta la aplicación desde el archivo ejecutable o desde el entorno de desarrollo. Al abrirse, se muestra una ventana principal con varias pestañas.

En la pestaña de tipos se puede crear un nuevo tipo, modificarlo, eliminarlo o listar los tipos existentes.

En la pestaña de categorías se pueden administrar las categorías que luego se asocian a los ítems.

En la pestaña de personas se registran los usuarios que pueden recibir préstamos. Para cada persona se indica su nombre, teléfono y correo electrónico.

En la pestaña de ítems se registran los objetos prestables. Para crear un ítem se indica el código, nombre, descripción, tipo y categorías. El tipo debe existir previamente, y las categorías indicadas también deben existir si se escriben.

En la pestaña de préstamos se puede crear un préstamo indicando el correo de la persona, los códigos de los ítems, los datos de la alerta y la fecha correspondiente. El sistema valida que los ítems no estén prestados antes de permitir el préstamo.

También se puede retornar un ítem específico, eliminar un ítem de un préstamo o finalizar un préstamo completo.

En la pestaña de reportes se pueden generar reportes por usuario, por ítem, por categoría y por tipo. Estos reportes muestran el estado de los elementos registrados y permiten revisar la información de manera ordenada.

Finalmente, se puede cerrar el programa y volverlo a abrir para comprobar que los datos se conservan gracias al archivo de persistencia.

f. Conclusiones

- Se concluye que la separación entre lógica, control e interfaz gráfica permitió organizar mejor el desarrollo del sistema. Esta división ayudó a que cada parte del programa tuviera una responsabilidad específica.
- Se concluye que la programación orientada a objetos permitió representar adecuadamente los elementos del problema. Las clases Tipo, Categoría, Persona, Ítem, Préstamo y Alerta permitieron modelar los datos principales del sistema.
- Se concluye que el uso de encapsulamiento fue importante para proteger los atributos de las clases y controlar el acceso a la información mediante métodos.
- Se concluye que la clase PréstamoControlador facilitó la comunicación entre la interfaz gráfica y la lógica del sistema, evitando que la interfaz tuviera que manipular directamente todas las listas y relaciones internas.
- Se concluye que el uso de ArrayList fue adecuado para este proyecto, ya que permitió manejar colecciones dinámicas de datos sin tener que definir un tamaño fijo.

- Se concluye que la relación entre préstamos e ítems fue fundamental para controlar la disponibilidad de los objetos y evitar que un ítem fuera prestado dos veces al mismo tiempo.
- Se concluye que los reportes permiten consultar la información del sistema de forma más clara, ya que agrupan los datos según usuario, ítem, categoría o tipo.
- Se concluye que la interfaz gráfica facilita el uso del sistema en comparación con una aplicación por consola, ya que permite acceder a las funcionalidades mediante ventanas, botones y campos de texto.
- Se concluye que la serialización permitió resolver la persistencia de datos de forma sencilla para un proyecto académico, ya que la información se guarda al cerrar el programa y se carga al abrirlo nuevamente.
- Se concluye que las validaciones son necesarias para mantener la consistencia de los datos, especialmente al crear ítems, crear préstamos y eliminar registros relacionados.

g. Recomendaciones

- Se recomienda mantener separadas las capas de lógica, control e interfaz en futuros proyectos, ya que esto facilita el mantenimiento y la comprensión del código.
- Se recomienda que la interfaz gráfica se comunique principalmente con el controlador, y no directamente con todas las clases lógicas, para mantener una mejor organización del sistema.
- Se recomienda mejorar la interfaz utilizando componentes como tablas, listas y cajas desplegables, con el fin de hacer el programa más fácil de usar.

- Se recomienda agregar más validaciones en los campos de entrada, por ejemplo validar formatos de correo, números de teléfono, fechas y campos vacíos.
- Se recomienda utilizar una clase especializada para el manejo de archivos en futuras versiones, de manera que la persistencia quede separada del controlador principal.
- Se recomienda considerar el uso de una base de datos si el sistema creciera, ya que permitiría manejar mayor cantidad de información de forma más segura y ordenada.
- Se recomienda asignar un identificador único a cada préstamo para evitar depender de la posición del préstamo dentro de una lista.
- Se recomienda utilizar clases de fecha como `LocalDate` para manejar fechas de préstamo y alertas de forma más correcta.
- Se recomienda ampliar el funcionamiento de las alertas para que el sistema pueda calcular cuándo deben activarse y mostrar avisos al usuario al iniciar la aplicación.
- Se recomienda documentar los métodos principales del sistema, especialmente los del controlador, para facilitar futuras modificaciones o revisiones del proyecto.